What Hinders Reinforcement Learning More: Action Noise, Observation Noise, or Reward Noise?

Azeez Adeniyi
University of Alberta
azeez1@ualberta.ca

Isaiah Heidt
University of Alberta
iheidt@ualberta.ca

Abstract

This paper investigates the impact of noise on various reinforcement learning (RL) algorithms using a policy gradient approach. We explore both actor-critic and Monte Carlo methods, evaluating their ability to learn and adapt in environments with varying levels of noise introduced to actions, rewards, and observations. Our results demonstrate that noise tolerance differs across algorithms and environments. Our findings emphasize the significance of designing RL agents that are resilient to noise, particularly in real-world scenarios where uncertainty is prevalent.

1 Introduction

Environmental noise is a major disruptor in deploying RL algorithms to solve real-world tasks. Many current state-of-the-art RL methods do not take into account the possibility of noisy environments. It is easy to find examples of noise in real-world environments. In robotics, small inaccuracies in robotic limb movements caused by physical wear can be seen as action noise. Errors in computer vision caused by poor lighting or occlusions can be seen as observation noise. Inaccurate and inconsistent human feedback on prompts provided by large language models can be seen as reward noise. Due to the inherent environmental noise in real-world tasks, it is critical to analyze and understand how different types of noise affect RL algorithms. Our project aims to explore this problem and provide motivation for further analysis of this topic.

2 Methodology

Noise evaluation. To assess the impacts of noise on RL, we analyze action, reward, and observation noise. Action noise is introduced by varying the probability $a \in [0, 1]$ that the agent takes a random action in the environment, the agent still observes that it took the action it initially chose. Observation noise is introduced in a similar fashion, we vary the probability $o \in [0, 1]$ that the agent observes it is in a random state within the environment. It is important to note that the agent is still in its true state, it only observes a random state. Reward noise is introduced by adding a $\mathcal{N}(0, \sigma^2)$ noise to all rewards received by the agent; we vary the level of reward noise by changing the standard deviation, σ, of the Gaussian distribution.

Algorithmic evaluation. We implement the n-step actor-critic algorithm as shown in . We run experiments analyzing the impacts of noise on TD updates equal to: $n = 1, n = 2, n = 3, n = 10$, and a full Monte Carlo roll-out where we wait until episode termination or truncation.

Environment evaluation. We run our experiments across four Gym compatible environments: the 3v3 Penalty gridworld, the Danger Maze gridworld, and the Cart Pole, and Mountain Car environments. The gridworld environments have discrete state and action spaces, whereas the Cart Pole and Mountain Car environments have discrete action spaces and continuous state spaces. A summary of the environments is shown in Table 1. For full implementations and details regarding these

Table 1: Environment specifications

Environment	Action space	State space
3v3 Penalty	Discrete	Discrete
Danger Maze	Discrete	Discrete
Cart Pole	Discrete	Continuous
Mountain Car	Discrete	Continuous

environments, please see `https://github.com/Farama-Foundation/Gymnasium/blob/main` and `https://github.com/sparisi/gym_gridworlds/blob/main`.

Experimental techniques. The metric we use to analyze performance is the expected returns achieved by different n-step roll-outs under noise, compared to baseline performances without noise. For each setting, we ran 50 seeds and 600,000 environment steps.

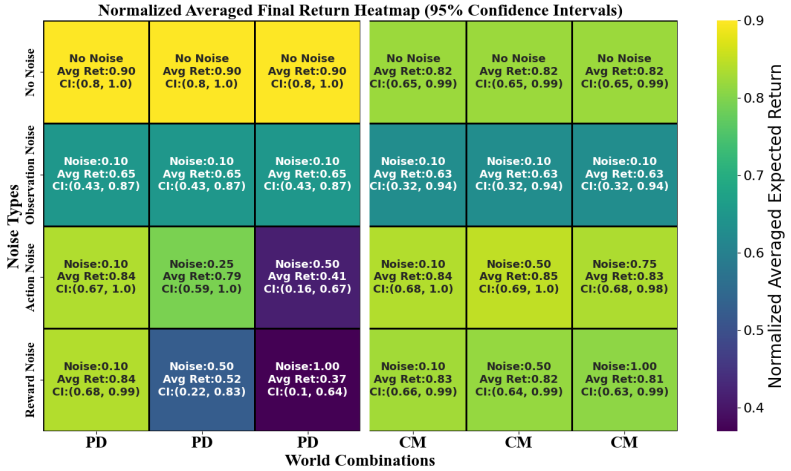


Figure 1: **Heatmap of aggregated results.** Returns are normalized to $[0, 1]$ using the maximum and minimum values achievable in each environment. They are then averaged over all n-step results and all 50 seeds for each experiment. The first 3 columns are additionally averaged over the 3v3 Penalty and Danger Maze gridworlds, whereas the final 3 columns are averaged over the Cart Pole and Mountain Car environments.

3 Results

3.1 Effect of noise on expected return

Figure 1 shows an overview of our main results. We see a stark difference between the effects of action and reward noise between the gridworlds and the Cart Pole and Mountain Car environments.

Action noise has a drastic negative impact on performance in the gridworlds but has a positive impact on performance in the Cart Pole and Mountain Car environments. We theorize that these differences are due to what we call the "riskiness" of the environments. In the gridworlds, there are many dangerous states. These dangerous states are accessible from nearly every state within the environment. Thus, a random action can easily transfer us into one of these penalizing states, greatly hindering the expected return. In the Cart Pole and Mountain Car environments, a single bad action has a much lower chance of harming us. In Cart Pole, a single bad action cannot lead to failure unless the pole is already in a vulnerable position close to parallel. In Mountain Car, a single bad action cannot lead to immediate failure, it may just cause us to lose our momentum. Thus, we believe action noise in the less "risky" settings of Cart Pole and Mountain Car adds the benefit of additional exploratory behavior with little downside.

Reward noise has a similar negative effect on the gridworld environments but seems to have little to no effect on the Cart Pole and Mountain Car environments. We attribute this difference to the sparsity of reward in these environments. In the gridworlds, rewards are very sparse, the only non-zero signals received by the agents are when they encounter a penalty state or choose to stay in the goal state. Thus, we believe reward noise can greatly confuse agents in these environments as they do not receive adequate feedback as to what behavior is considered good or bad. On the other hand, rewards are very non-sparse in the Cart Pole and Mountain Car environments. Every transition, some non-zero reward signal is provided to the agent. We believe that this leads the agent to be less confused by noisy rewards, able to use the large amounts of feedback to still determine reasonable behavior.

Observation noise is devastating across all the environments, greatly reducing performance even at low levels. Thus, we claim that observation noise generally has the most negative impact among the noise variants considered in this project.

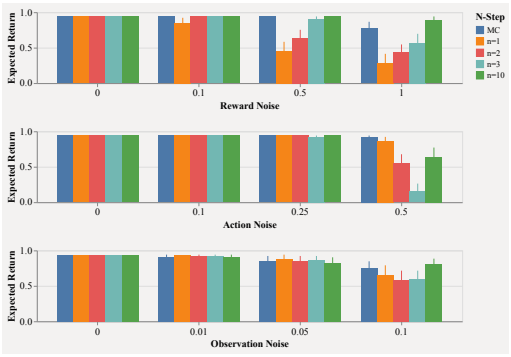


Figure 2: 3v3-Penalty

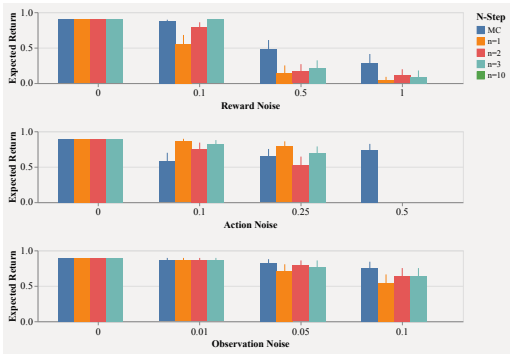


Figure 3: Danger Maze

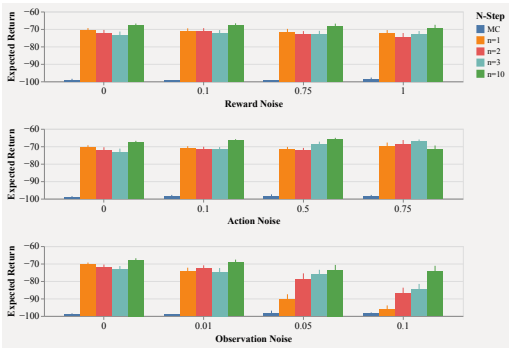


Figure 4: Mountain Car

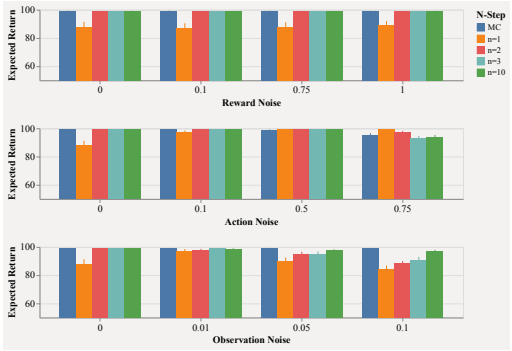


Figure 5: Cart Pole

Figure 6: Final returns achieved by different algorithms under varying noise levels.

3.2 Effect of noise on different n-steps

Here, we analyze how different n-step roll-outs are impacted by noise. Figure 6 shows the final returns achieved across n-step algorithms in each environment. Note that the Monte Carlo and the $n = 10$ roll-outs are unable to learn in the Mountain Car and Danger Maze environments, respectively, even without noise. In the Danger Maze, $n = 10$ may struggle due to issues in balancing bias and variance. Lower n-values benefit from decreased variance at the expense of increased bias, whereas Monte Carlo benefits from decreased bias at the expense of increased variance. In the Danger Maze, it appears $n = 10$ causes the agent to suffer from both an increase in bias and variance, struggling to learn. In the Mountain Car environment, episodes are very long, causing Monte Carlo to struggle as it must wait until the end of an episode to perform updates.

Under high **reward noise** in the gridworlds, longer n-steps are generally more robust. The goal of our RL algorithms is to maximize the sum of expected rewards. Introducing $\mathcal{N}(0, \sigma^2)$ noise does not change this expectation. Hence, we would expect this noise to have little effect. Lower n-step

roll-outs are a coarser approximation to the sum of expected rewards, and thus appear to be more affected by reward noise in the gridworlds. Monte Carlo and $n = 10$ more closely approximate the sum of expected rewards, and thus, the reward noise is more accurately "averaged out" in a sense.

Under **action noise**, Monte Carlo is more robust in the gridworlds whereas lower n-values seem to marginally perform better in the Cart Pole and Mountain Car environments. Lower n-values allow the agent to learn more frequently, but on a smaller set of sample returns. Larger n-values cause the agent to learn less frequently, but from a larger set of sample returns. The differences in n-value performance under action noise may again come from what we described as the "riskiness" of the environments. In the riskier gridworlds, a single bad action can lead to a poor reward. For low n-values, the agents will react quickly to seeing a poor reward, possibly hindering performance when they mistakenly change their beliefs. For larger n-values, the agent sees more and does not immediately react to a poor reward received from a noisy action. This patience may allow the agent to better overcome the random actions in these riskier gridworld settings. In the less risky Cart Pole and Mountain Car environments, random actions do not often result in poor rewards. Thus, the lower n-values are able to more quickly take into account the exploratory benefits of the random actions, where the larger n-values may cause the agents to be too patient to fully reap the exploratory benefits.

Under **observation noise**, larger n-values are more robust across all the environments. We believe this is due to the larger sample sets used and less frequent updates made by larger n-value algorithms. With smaller n-values, the agents see a smaller set of sample returns and react more quickly based on this limited information. Thus, a noisy observation is more strongly and quickly reacted to, changing the agent's beliefs and future behavior based on a noisy observation. For example, for $n = 1$, if an agent experiences a noisy observation, this is all the information they use to make their updates, fully believing they are in a false state. For larger n-values, agents are more patient, naturally waiting to see more of the environment through a larger sample set before altering their behavior. With this larger sample set, each update is made based on more information, leading to a lower chance of reacting drastically to one noisy observation.

4 Conclusion

Our results have the potential for influencing how we choose our algorithms and how we design our RL models. When dealing with action noise, performance is greatly hindered when a single bad action can lead to failure. Thus, if we design our agents to take smaller, more conservative actions, it may lead to better performance under action noise. When dealing with reward noise, performance is strongly hindered in sparse reward settings. Thus, we should try and find ways to provide more feedback to agents to minimize confusion. It appears to be best to avoid observation noise, as it was negatively impactful in all environments. However, when dealing with observation noise, using an algorithm with longer n-step roll-outs may help the agent to perform better.

5 Limitations and future work

The major limitations of our methodology are the lack of hyperparameter tuning, the simplicity of the environments we use, and that our implementation of reward noise is not directly comparable to action or observation noise. Possibilities for future work include: i) testing different learning rates, ii) considering state-value or action-value algorithms, iii) analyzing the effects of combining different noises, and iv) testing algorithms under noise on more complex and continuous action space environments.

6 Acknowledgments

We wish to acknowledge and thank Simone Parisi for all the help and guidance provided throughout this project. Additionally, we acknowledge the Digital Research Alliance of Canada for providing the computational resources required to run our experiments and obtain our results.

Appendix

A Effect of noises on actor θ and critics δ and w

Besides looking at the agent's ability to maximize the expected return, we will consider other metrics like the δ, variance in policy (θ), and variance in critic weights (w). Since our methodology adopts an actor-critic approach, we also look at the change in the critic's δ value, which serves as a critic signal to the actor. This signal indicates whether the taken action should be kept or discarded. We note that the δ value from Monte Carlo methods may be too dominant, as it cannot be strictly called actor-critic due to the lack of bootstrapping in its computed δ.
There are several reasons why we consider the θ and w variances. A high variance in θ and w indicates that updates to the parameters are fluctuating widely, which can potentially cause instability in the learning process. High variance values could reflect a failure to converge the policy and value function, leading to oscillations or divergence. A high value of θ may cause overly aggressive or erratic exploration, making it difficult for the agent to balance exploration and exploitation effectively. With unchecked increases in θ, the agent's policy can degrade, causing it to frequently select suboptimal or random actions. A high value of variance in w can destabilize both the value function and policy updates by amplifying errors in value estimates, leading to large, inconsistent TD errors. Serious fluctuations in both the θ and w variances could imply instability in the learning process, poor convergence, and reduced generalization.

In the Penalty gridworld, all the methods perform equally well when there was no noise but method n-step 1 and 2 are accompanied with relatively higher θ compared to others (Supplementary Figure S7) which is an indication that there was a struggle in their learning. This is the reason why adding reward noise significantly affected both n-step 1 and 2 compared to others. Action noise is preferable to reward noise in the Penalty world as 0.1 and 0.25 when considering the expected return but they are also accompanied with a higher price of relatively higher variance in both w and θ (Figure S7, S11). MC and n-step 10 appeared to be more tolerant to the noises in the Penalty world than other methods.

In Dangermaize, n-step 10 did not learn during any of the experiments while others have similar expected returns when there was no noise. Though the analysis of variance in weights shows that MC and n-step 10 experience higher instability in state value, this does not affect their action policy as they are characterized with lower variance in θ (Figure S8). At 0.5 action noise, the variance in the w became erratic, indicating an unstable state value due to an inability to balance exploration and exploitation, leading to all the methods failing completely to learn except MC, which is still characterized with lower variance in w. All the noise negatively affects all the methods in the Dangermaize world compared to the Penalty world (comparing Figure S8 and S7) as characterized with relatively higher variance in w and θ. MC appears to be more tolerant to the noises in Dangermaize than other methods.

In Mountaincar, the n-step 10 outperforms all other algorithms according to the expectation and is also accompanied by higher δ that encourages the chosen action. It is also characterized with lower variance in w and θ indicating stability and balancing of the exploration and exploitation even after the noises (Figures S9). Though n-step 1, 2, and 3 try to compete with n-step 10 in terms of the expected returns, they are characterized with huge variance in w and relatively higher θ indicating destabilization of the value function and policy updates, especially n-step 1 (S9, S13). At the same time, the w near zero for MC is an indication that there was no learning, as obvious from the detail plots added to the supplementary Figure S13. The action noise of 0.95 leads to a very low δ, which is an indication less rewarding action. But strange that it is helpful to the methods n-step 1, 2, and 3 that outperform n-step 10 and are characterized with stability leading to lower variance in both their w and θ (Figure S9). N-step 10 appeared to be more tolerant to the noises in the Penalty world than other methods.

A better learning was observed in Cartpole compared to Mountaincar as obvious from the expected return and is also accompanied by better delta, lower levels of variance in the w, an indication of stability in learning (comparing Figure S10 and S14 to S9 and S13). The MC and n-step 10 outperform others in most of the scenarios. The n-step 1 performs generally poorly compared to other algorithms in terms of expected return and is also accompanied by relatively higher variance in both w and θ (S10). As the action noise of 0.1 and 0.5 improves the performance of n-step 1, it also resulted in stability in exploration and exploitation as accompanied by lower θ compared to no noise,

but a noise of 0.95 decreases the performance and leads to instability in learning that leads to a very high variance in w (S10). MC and n-step 10 appeared to be more tolerant to the noises in Cartpole than other methods.

B Algorithms

Algorithm 1 One-step Actor-Critic (episodic)

Initialize policy parameters θ and value function weights w
for each episode **do**
  Initialize S
  while S is not terminal **do**
    Take action $A \sim \pi(\cdot|S, \theta)$
    Observe reward R and next state S'
    (If S' is terminal, then $v(S', w) = 0$)

$$\delta = R + \gamma v(S', w) - v(S, w)$$

    Update value function:

$$w \leftarrow w + \alpha_w \delta \nabla_w v(S, w)$$

    Update policy:

$$\theta \leftarrow \theta + \alpha_\theta I \delta \nabla_\theta \log \pi(A \mid S, \theta)$$

    $I \leftarrow \gamma I$
    $S \leftarrow S'$
  end while
end for

Algorithm 2 n-step TD for Estimating V^π

Require: Policy π, step size $\alpha \in (0, 1]$, discount factor $\gamma \in [0, 1]$, number of steps n
Initialize $V(s)$ for all $s \in \mathcal{S}$ arbitrarily, except $V(terminal) = 0$
for each episode **do**
  Initialize S_0
  $T \leftarrow \infty$, $t \leftarrow 0$
  while $\tau < T - 1$ **do**
    if $t < T$ **then**
      Take action according to policy $\pi(\cdot|S_t)$
      Observe R_{t+1} and S_{t+1}
      if S_{t+1} is terminal **then**
        $T \leftarrow t + 1$
      end if
    end if
    $\tau \leftarrow t - n + 1$
    if $\tau \geq 0$ **then**
      $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
      if $\tau + n < T$ **then**
        $G \leftarrow G + \gamma^n V(S_{\tau+n})$
      end if
      $V(S_\tau) \leftarrow V(S_\tau) + \alpha \left(G - V(S_\tau)\right)$
    end if
    $t \leftarrow t + 1$
  end while
end for

C Supplementary graphs

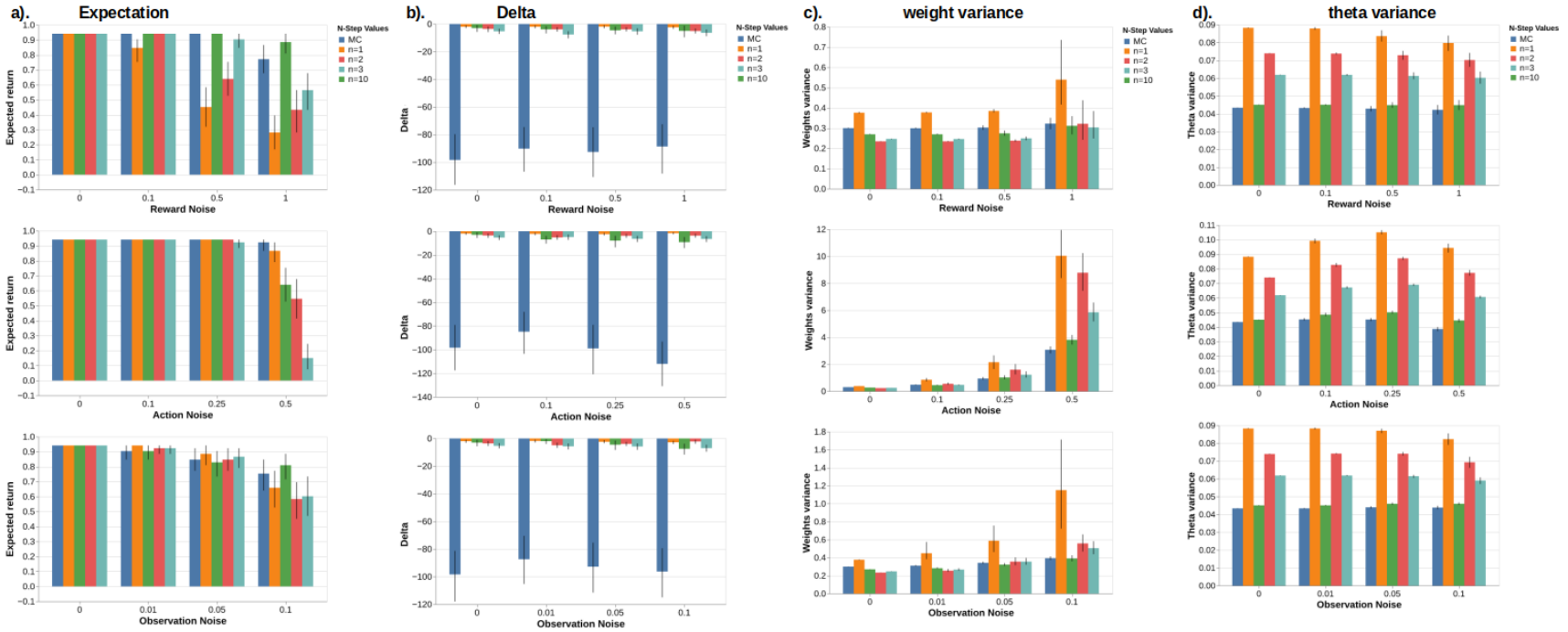


Figure S7: The average values of Expected return, critics delta, critic weight variance and actor theta variance for the Penalty 3x3 gridworld environment showing effects of different level of noises

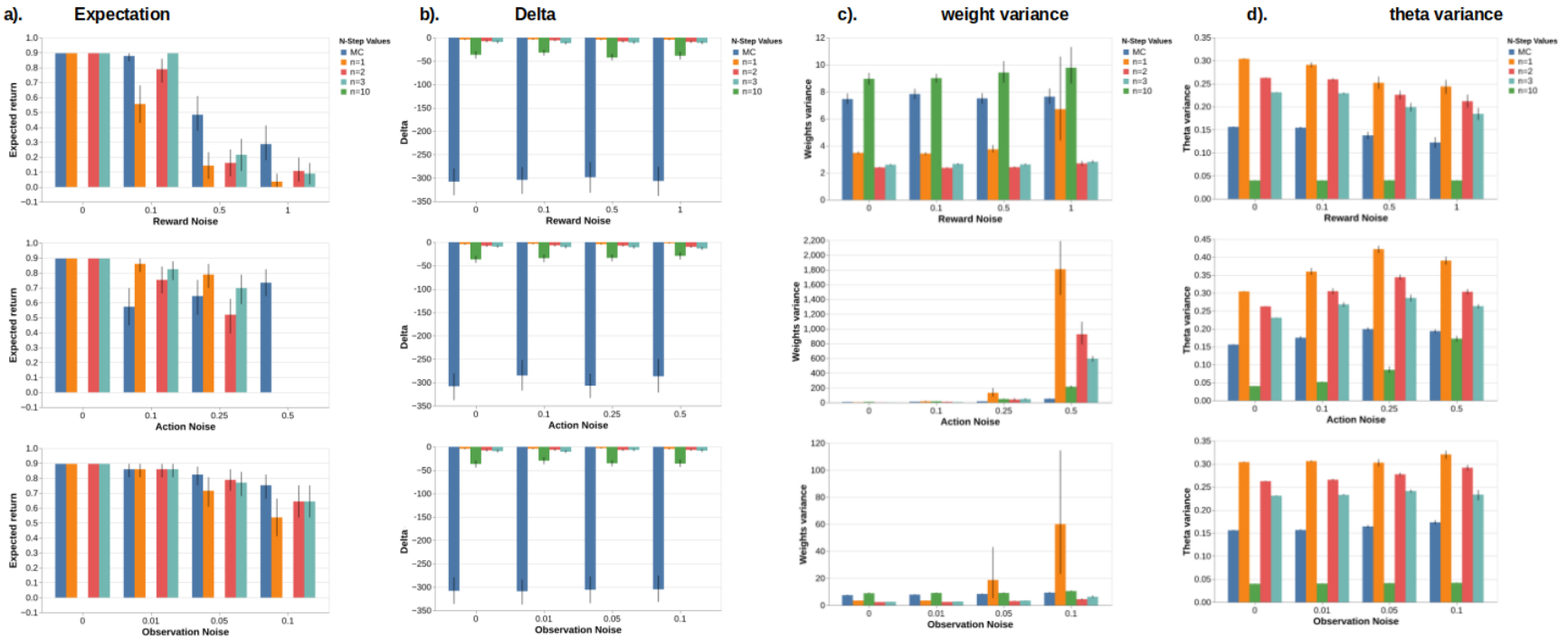


Figure S8: The average values of Expected return, critics delta, critic weight variance and actor theta variance for the Dangermaize gridworld environment showing effects of different level of noises

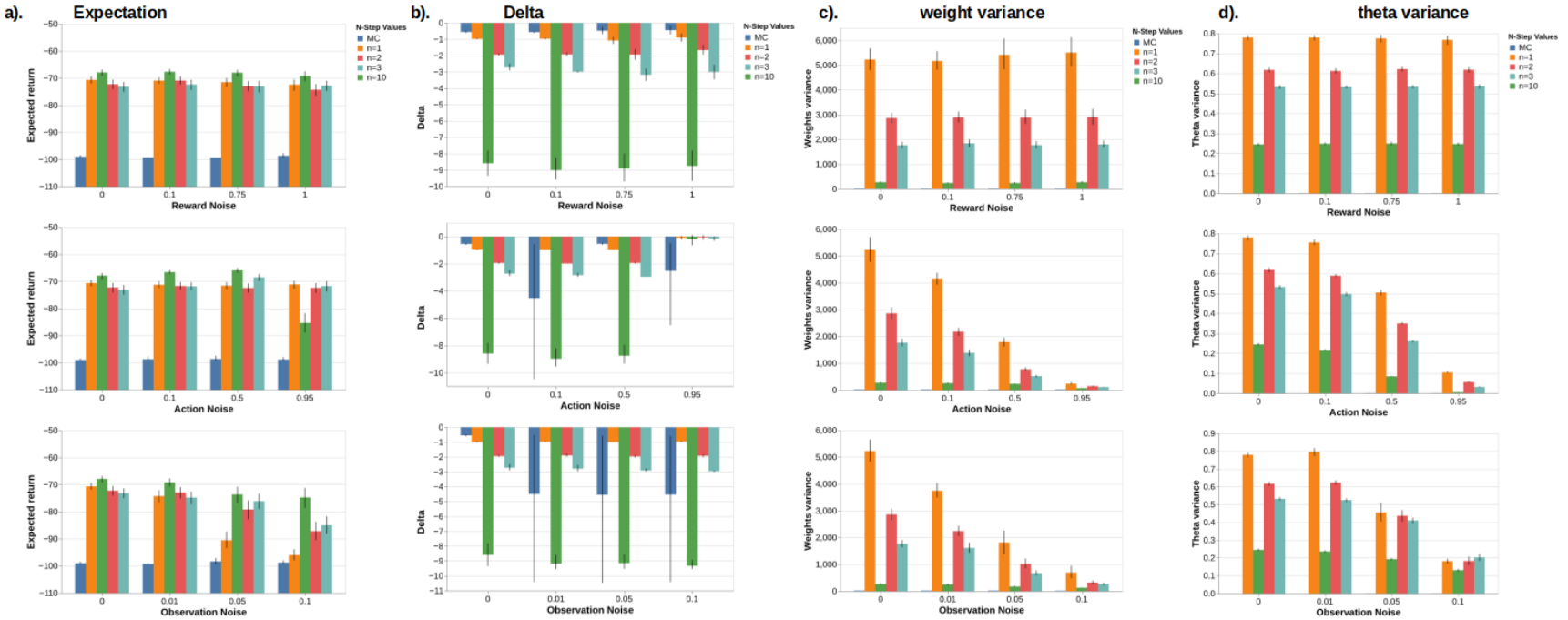


Figure S9: The average values of Expected return, critics delta, critic weight variance and actor theta variance for the Mountaincar environment showing effects of different level of noises

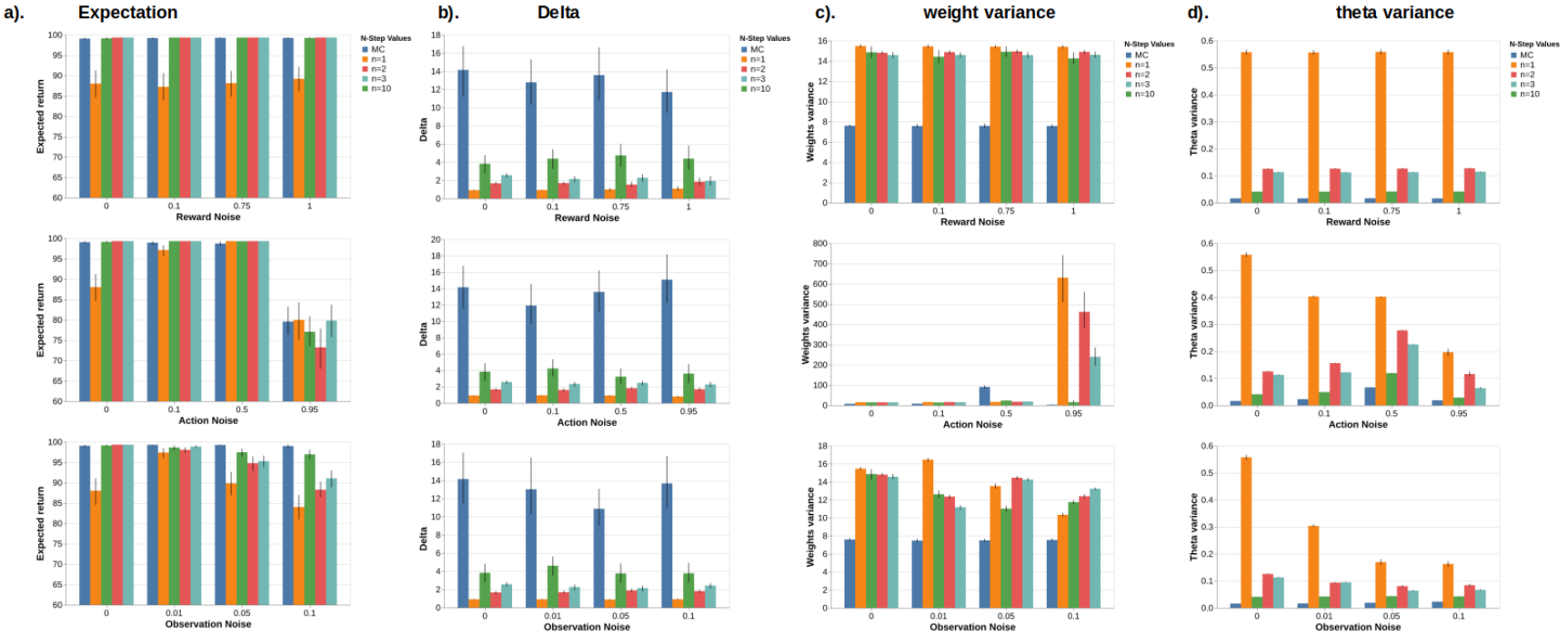


Figure S10: The average values of Expected return, critics delta, critic weight variance and actor theta variance for the Cartpole environment showing effects of different level of noises

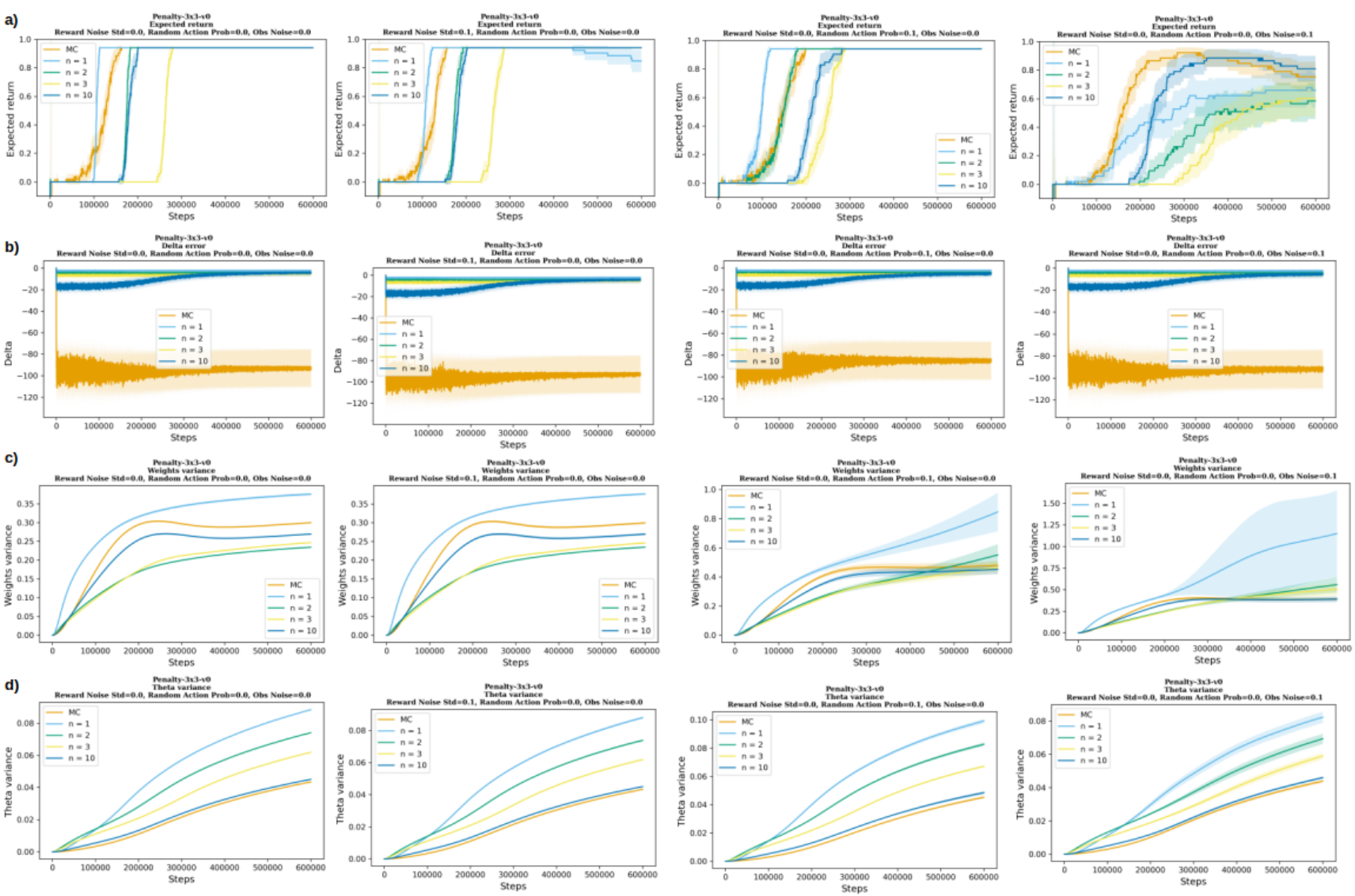


Figure S11: Effect of 0.1 noises on the expected return, critics delta, critic weight variance and actor theta variance for the Penalty 3x3 gridworld environment

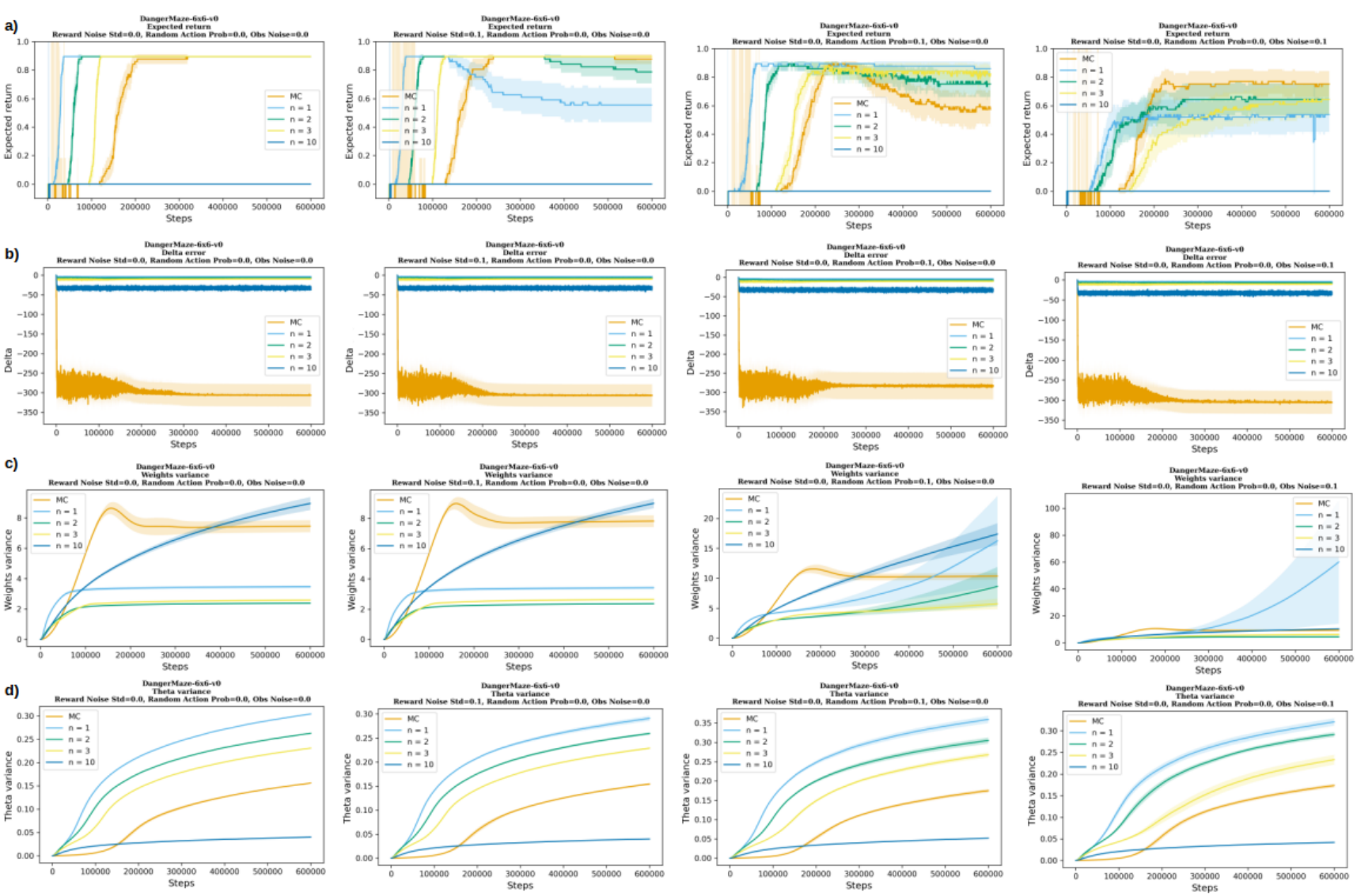


Figure S12: Effect of 0.1 noises on the expected return, critics delta, critic weight variance and actor theta variance for the Dangermaize gridworld environment

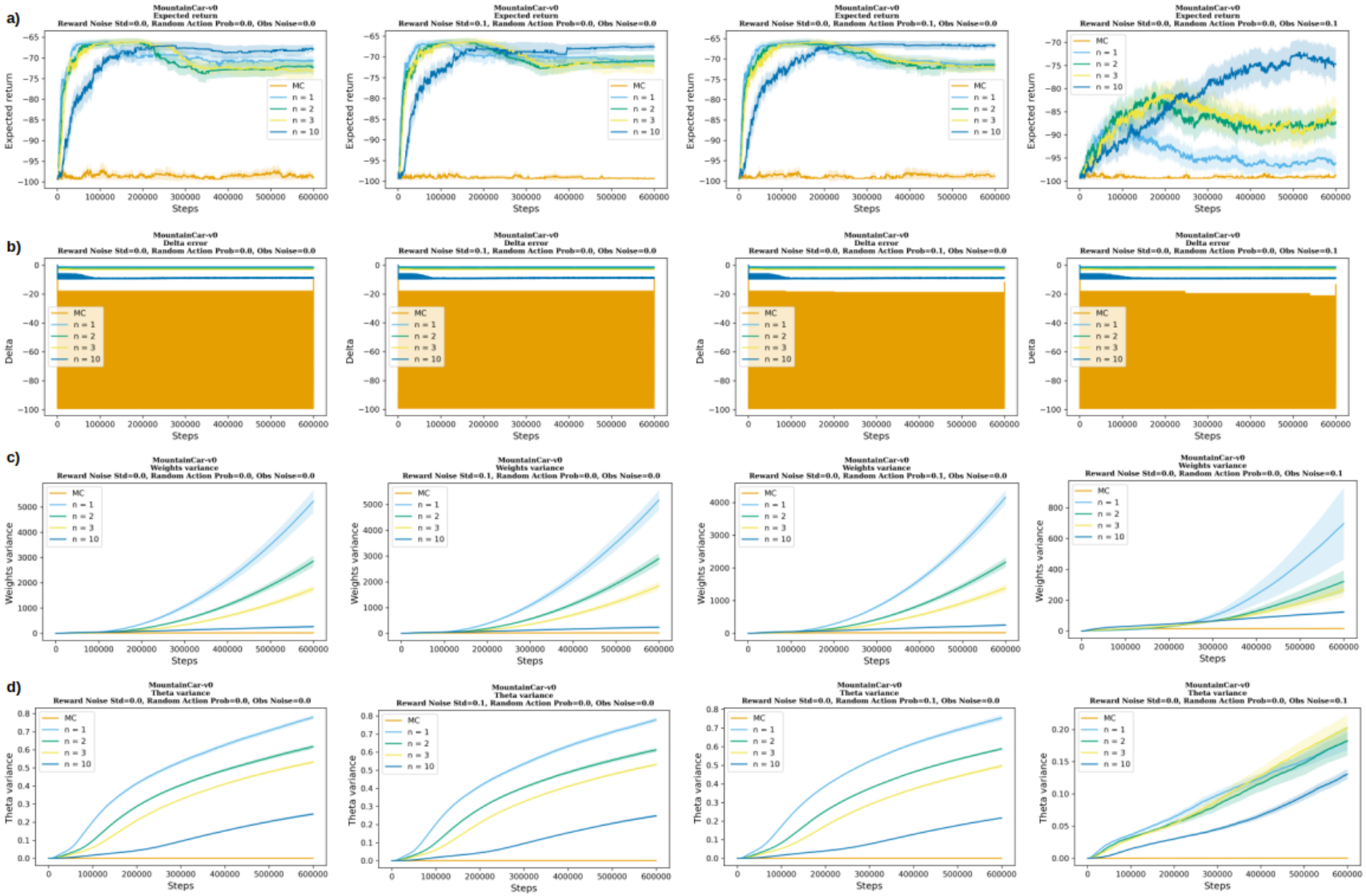


Figure S13: Effect of 0.1 noises on the expected return, critics delta, critic weight variance and actor theta variance for the Mountaincar environment

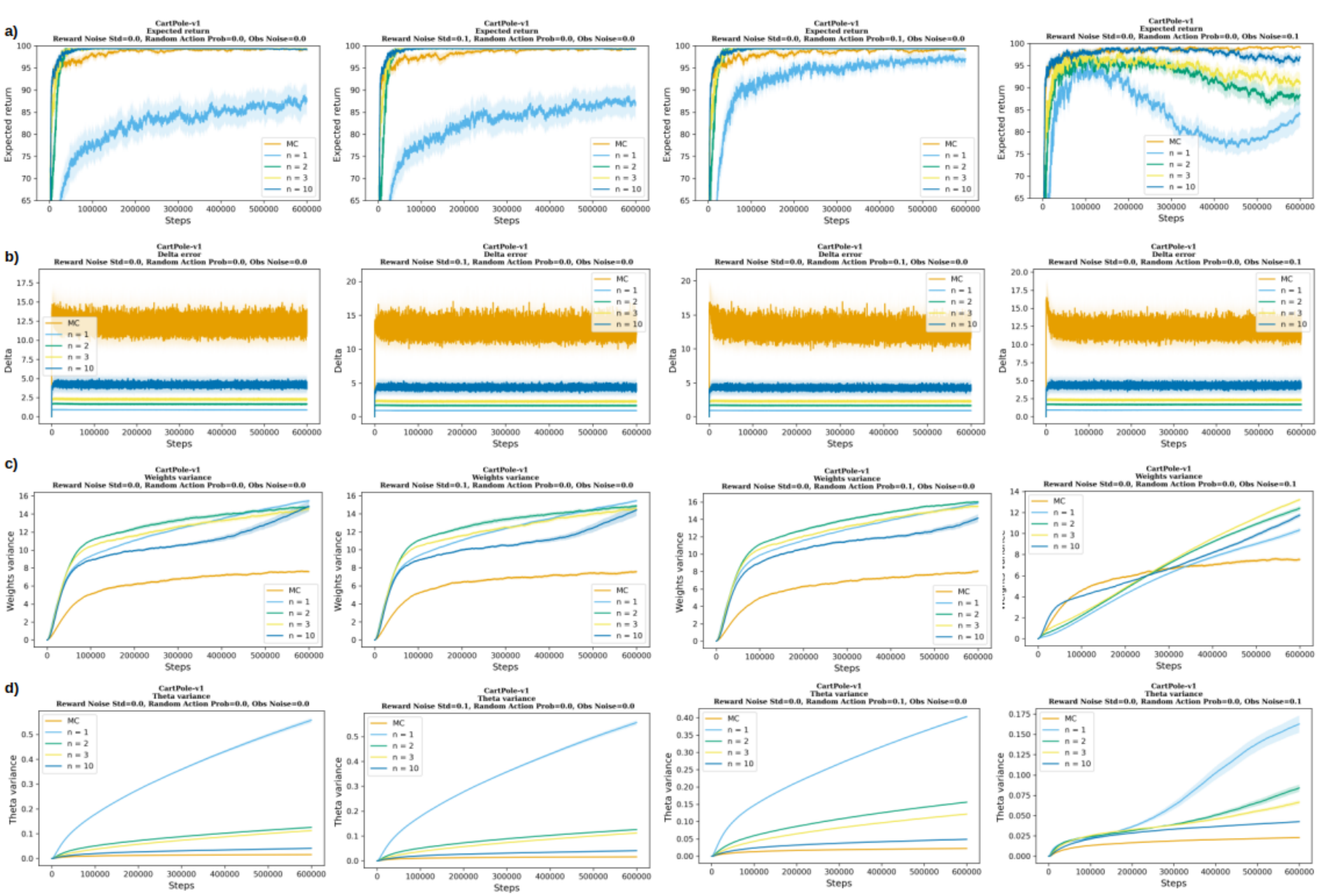


Figure S14: Effect of 0.1 noises on the expected return, critics delta, critic weight variance and actor theta variance for the Carpole environment